

Control de flujo

No.	Respuesta	Explicación:
1	b. Se puede utilizar con cualquier tipo de colección y matriz.	El for-each se puede usar para recorrer arreglos y colecciones que implementan Iterable. Por eso no está limitado solamente a matrices. Además, no es recomendable modificar agregando o eliminando elementos de una colección mientras se recorre porque puede producir errores.
2	c. 0	a vale 10 y b vale 37. Como 10 no es igual a 37 y tampoco es mayor, ninguna de las condiciones del if se cumple. Por eso z mantiene su valor inicial 0 y después $w = 10 * 0$, dando 0.
3	b. Count: 5	El for va desde 0 hasta 9 y el if cuenta los números pares. Los pares son 0, 2, 4, 6 y 8, así que tenemos cinco números pares.
4	a.i.1,j.1 i.1,j.2 i.2,j.1 i.2,j.2 i.3,j.1 i.3,j.2	Aquí tenemos un for dentro de otro for. Primero i vale 1 y el segundo ciclo hace completamente $j = 1$ y $j = 2$. Después i pasa a 2 y vuelve a hacer los dos valores de j, y finalmente ocurre lo mismo con $i = 3$. Por eso sale este resultado.
5	c. 24	Como tenemos tres ciclos anidados, tenemos que multiplicar sus iteraciones: $4 \times 3 \times 2 = 24$. El ciclo de i se ejecuta 4 veces, el de j 3 veces y el de k 2 veces. Por eso el println se ejecuta 24 veces.
6	d. An infinite loop	ii empieza en 2 y dentro del do nunca cambia. Entonces siempre se imprime 2 y la condición $ii > 0$ siempre sigue siendo verdadera. Como nunca cambia el valor de ii, el ciclo nunca termina.
7	b. Compilation fails.	No compila ya que la condición de un while debe ser un boolean. --ii devuelve un int, no un boolean.
8	e. 11	Si empezamos con $ii = 11$, el ciclo pasa por 11, 10, 9, 8 y 7, que son cinco vueltas. Cuando ii llega a 6, la condición $ii > 6$ ya es falsa y el ciclo termina. Por eso jj aumenta exactamente cinco veces y queda en 5.
9	b. 3	El programa suma los valores cuando el índice i es par y los resta cuando es impar. Entonces tenemos $1 - 2 + 3 - 4 + 5$, que da 3.
10	a. The fox lazy...	Como 33 no es 0, se imprime the. Después $33 < 10$ es falso. En el siguiente if, $33 > 30$ es verdadero, por lo que imprime fox y ya no revisa los otros else if. Después $33 > 10$ es verdadero y se imprime lazy, y finalmente se imprime "..."
11	c. Found Red Found Blue Found White.	El switch encuentra "Red", pero después de ese case no hay break. Entonces Java continúa ejecutando los siguientes case,

		primero "Blue" y después "White". Cuando llega al break de "White", sale del switch y no llega al default.
12	e. 1111	El switch entra en TUESDAY, pero como no hay ningún break, continúa con WEDNESDAY y THURSDAY. Empezamos con 1, luego sumamos 10, después 100 y finalmente 1000: $1 + 10 + 100 + 1000 = 1111$.
13	f. BCF	grade vale 'B', por lo que entra en el case 'B' y primero imprime B. Como no hay break, continúa al case 'C' e imprime C, y después llega a case 'F' e imprime F. Por eso termina siendo BCF. Las variables a y d son final y tienen valores constantes, por lo que pueden utilizarse como case.
14	f. BC	grade vale 'B', así que imprime B. Después, como no hay break, continúa con C e imprime C. Pero después de C sí se tiene un break, así que el switch termina.
15	b. greater than - 6	x empieza en 0 y el x++ primero compara y después incrementa. El ciclo termina cuando se compara $5 < 5$, que es falso, pero aun así x ya aumentó a 6. Entonces $x > 5$ es verdadero y el mensaje es "greater than", por lo que se imprime greater than - 6.
16	e. Once	El for recorre tres elementos, pero dentro tenemos un while que imprime el valor de ii y después hace break. Se imprimen 0, 1 y 2, por lo que el número 2 aparece solamente una vez.
17	d. x = 4	La variable b es un boolean static y por defecto comienza en false. En else if (b = false) se está haciendo una asignación, no una comparación. Esa asignación deja b en false y la condición también resulta false. El siguiente else if (b) también es falso, así que finalmente se ejecuta el else y x queda en 4.
18	d. "foo".equals("bar")	La condición de un while necesita ser boolean. "foo".equals("bar") devuelve un boolean, en este caso false. compareTo() devuelve un int, i = 1 también trabaja con un int y el operador ternario de la opción c devuelve int, así que esas opciones no sirven como condición del while.
19	a. 12432432 The co value = 8	El for primero ejecuta test('1'), después revisa test('2'), entra al cuerpo y ejecuta test('4'), y al terminar ejecuta test('3'). Esto ocurre dos veces y después se vuelve a evaluar test('2') una última vez para comprobar que el ciclo terminó. Por eso queda 12432432 y se llamaron ocho veces al método test, así que co vale 8

20	d. <code>int x = 2, y = 3, z = 4;</code>	z controla cuántos números aparecen por línea, y como vemos 1 2 3 4, necesitamos <code>z = 4</code> . El <code>do-while</code> genera tres líneas antes de terminar, por eso <code>y = 3</code> . Finalmente aparecen dos bloques separados por <code>---</code> , así que el <code>for</code> debe ejecutarse dos veces y por eso <code>x = 2</code> .
----	--	---